Harness^2: Recursive Agent Harnessing for an Open World

Xu et al. · Northwestern University, Google Cloud AI Research · arXiv 2610.14224 (Oct 2026)

Paper Review

Key Terms

Harness	frozen model 바깥에서 agent를 동작시키는 scaffolding (prompt, memory, skills, tools, orchestration). 이 논문에서는 8개의 편집 가능한 component 파일로 구성.
RSI	Recursive Self-Improvement. 시스템이 실행 결과와 반성을 persistent한 변경으로 바꾸고, 그 변경이 이후 개선 방식에도 영향을 주는 것.
Harnessing	base model의 weight는 고정한 채, 주변 runtime scaffolding을 최적화하는 과정.
Task-level Recursion	현재 task의 execution harness를 Propose–Probe–Compose 단계로 refine. parallel / sequential로 확장 가능.
Harnessing-level Recursion	refinement 경험을 모아 improvement harness(library)를 update해서, 다음 task의 개선에 활용.

Motivation: Problem Statement

Agent = Model + Harness

같은 모델도 harness에 따라 성능이 크게 달라지고, harness를 사람이 매번 설계하는 것은 engineering 비용이 큼 → RSI로 자동 개선

Challenge 1: No Ground-Truth Verifier

open-world task는 harness 설계 시점에 보지 못한 새로운 task라 정답/검증기가 없음 → test time의 harness edit를 믿을 만한 신호로 guide하기 어려움

Challenge 2: Task-Specific Improvement

한 task에 맞춘 harness를 다음 task가 그대로 상속하면 noisy해짐. 반면 개선 과정(약점 진단, 시도한 변경, 실행 결과)은 재사용 가능한 경험인데 버려짐

Central Question: RSI로 현재 harness와, 미래 harness를 개선하는 방식을 모두 개선할 수 있는가?

Background: What is a Harness?

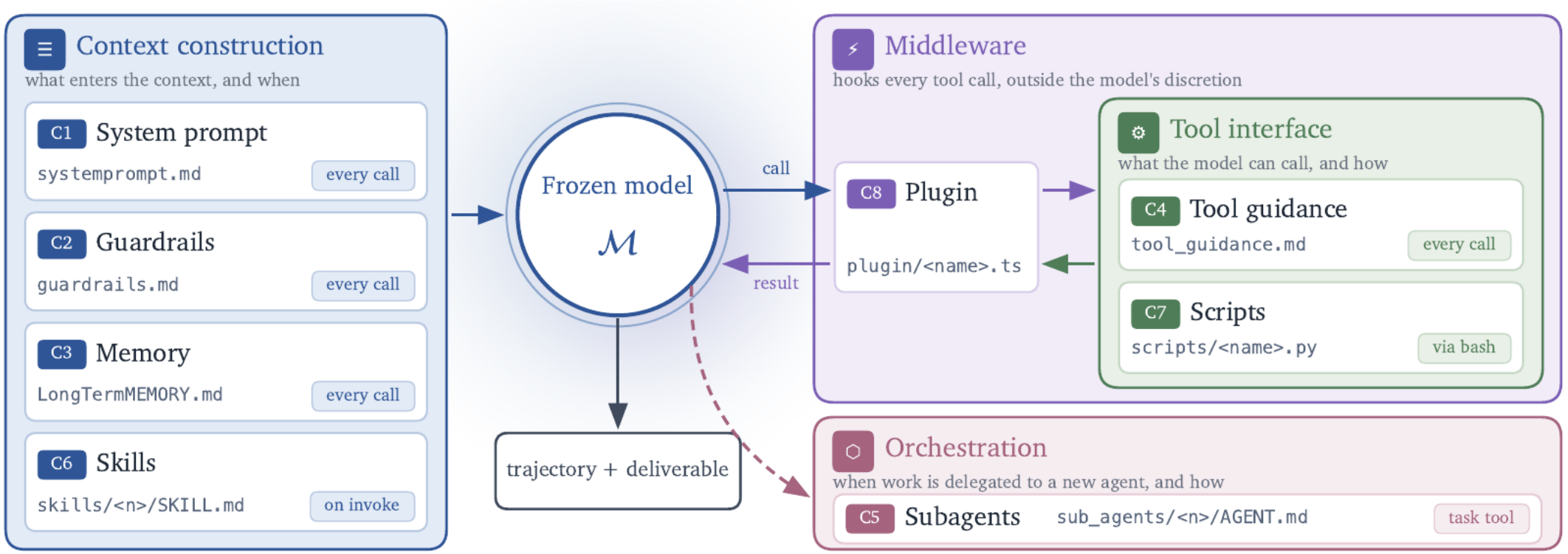


Figure 2. Harness²의 8개 harness component. model weight는 고정이고, 각 component는 runtime에 파일로 mount됨.

C1 System prompt · C2 Guardrails · C3 Memory · C4 Tool guidance · C5 Subagents · C6 Skills · C7 Scripts · C8 Plugin(middleware) — 이 파일들을 수정하는 것이 "harnessing"

Method: Two Coupled Levels of Recursion

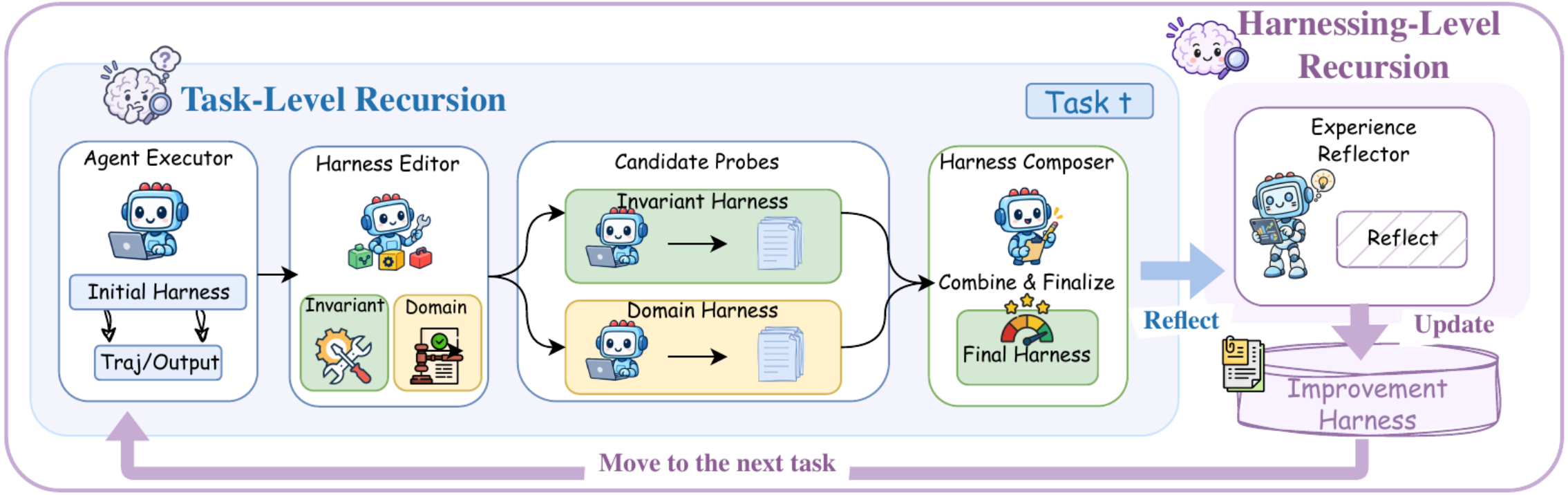


Figure 3. Harness² pipeline. 각 task 후 경험이 Improvement Harness에 쌓이고 다음 task로 이어짐.

Task-Level Recursion: Executor → Editor → Probes → Composer → Final Harness (정답 없이 현재 task의 harness를 개선)
Harnessing-Level Recursion: Reflector가 개선 과정에서 교훈을 추출해 Improvement Harness를 update (다음 task에 활용)

Method: Propose–Probe–Compose

Propose

- 현재 harness로 reference execution을 1회 수행
- invariant candidate: planning, tool use, context management, delegation 등 domain과 무관한 결함 대상
- domain-specific candidate: 현재 domain의 지식, 절차, 제약 대상
- library의 기존 component 재사용 가능 (task-specific fact는 현재 evidence로 확인)

Probe

- 각 candidate harness를 같은 task, 같은 model/environment 조건에서 실행
- reference와 trajectory, deliverable을 비교해 behavioral contrast를 얻음
- 의도한 결함이 완화됐는지, 변화가 없는지, regression이 생겼는지 확인

Compose

- reference, candidates, diff, 실행 evidence를 모두 읽고 successor harness 생성
- 지지된 변경만 merge, 충돌 시 수정, 후보 하나 채택, 또는 현재 harness 유지
- 각 결정은 구체적 관찰에 근거 (예상 점수 향상이 아님)

Probe의 contrast는 정답 여부를 검증하지 않음. 실행의 stochasticity와 shared model bias 때문에 correctness 근거가 아니라 composition을 위한 증거로만 사용.

Method: Scaling the Recursion

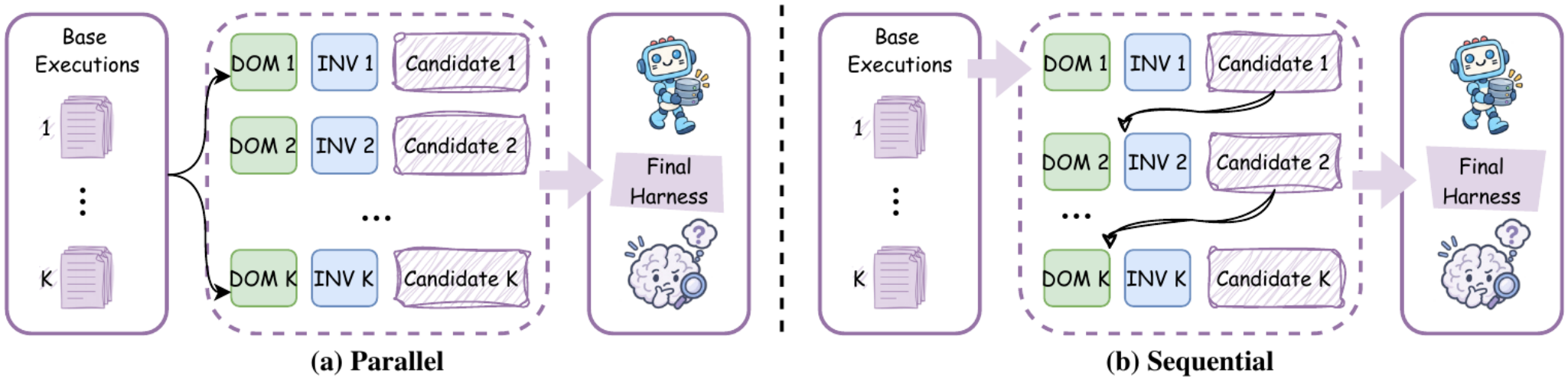


Figure 4. (a) Parallel recursion: k개 후보를 독립적으로 composition. (b) Sequential recursion: 개선된 harness와 evidence를 다음 step으로 전달.

Parallel vs. Sequential

- k는 탐색 budget. parallel은 폭, sequential은 깊이를 늘림. 최종 harness는 agentic selector가 익명화된 deliverable을 비교해 선택

Harnessing-Level Recursion

- task 후 improvement record를 reflect하고, 실행 contrast로 뒷받침된 교훈만 harness improvement library(harness + EXPERIENCE.md)에 저장
- 다음 task는 항상 공통 H_0에서 시작하고, library는 Propose와 Compose만 guide (이전 harness를 그대로 상속하지 않음)

Experimental Setup

Benchmarks

- WorkBuddy-Bench: office-style deliverable (Code, Web, Office)
- JobBench: 긴 실행이 필요한 end-to-end 직무 task
- LAB: 법률 업무 benchmark (Partial = criterion별 pass rate)

Executors & Editor

- frozen executor: Gemini 3.5 Flash, GLM-5
- initial harness: OpenCode, Codex (총 4개 executor–harness 구성)
- 기본 harness editor: Claude Opus 4.8

Baselines

- Base harness: 초기 harness 그대로
- ReasoningBank: harness 고정, 과거 전략을 retrieval
- Meta-Harness: training split으로 offline 탐색
- Harness Scaling: instance마다 추가 inference budget으로 harness 수정

Harness² "main" = 1회 improvement step (k=1), "+ Recursion" = parallel recursion k=5. improvement library는 benchmark stream마다 빈 상태로 시작하고, 결과는 3회 실행의 평균.

Results: Consistent Gains Across Models and Benchmarks

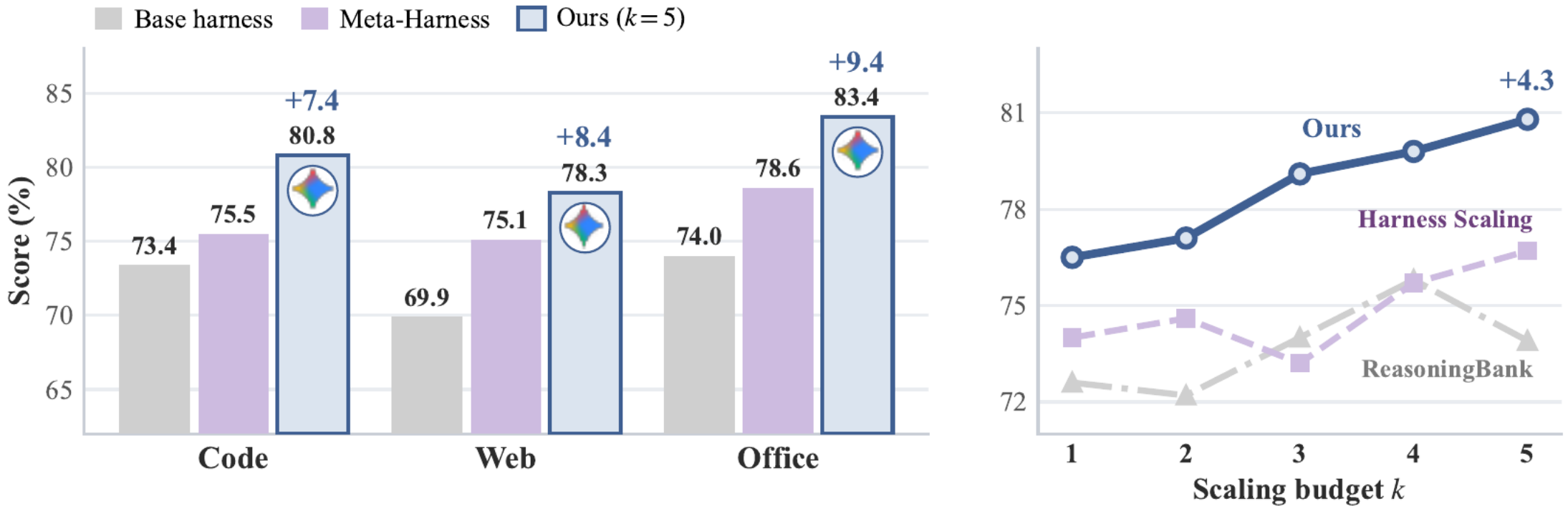


Figure 1. 왼쪽: WorkBuddy-Bench (Gemini 3.5 Flash, OpenCode). 오른쪽: WB-code에서 budget k에 따른 점수.

- 1회 improvement step으로 WorkBuddy-Bench 최대 +11.5, JobBench +4.5, LAB Partial +11.8%p. k=5 recursion은 추가로 최대 +3.9, +4.2, +6.7
- 약한 executor의 이득이 큼: 개선된 GLM-5(75.7)가 원래 harness의 Gemini(72.4)를 능가 (WorkBuddy-Bench, OpenCode)
- ReasoningBank는 coding에서 최대 6.6점 하락, Harness Scaling은 12개 조합 중 7개에서 base보다 낮음

Analysis: Ablation & Library Reuse

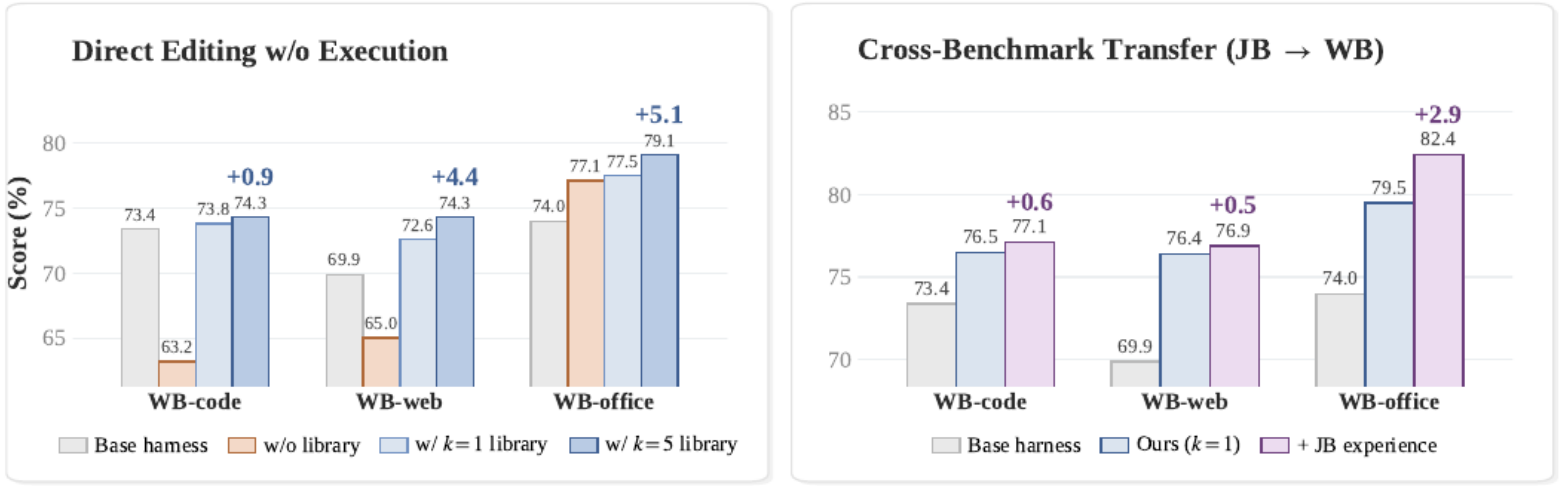


Figure 7. 왼쪽: 실행 없이 library로만 harness를 직접 수정. 오른쪽: JobBench에서 쌓은 library를 WorkBuddy-Bench로 transfer.

Ablation (WorkBuddy-Bench / JobBench 점수 하락)

- w/o library: −3.1 / −1.8 · w/o scoping: −2.9 / −3.7 · w/o composition: −1.6 / −1.9 (세 component 모두 기여)

Library & Transfer

- 실행 evidence 없이 base harness를 직접 고치면 오히려 하락 (Code 73.4→63.2). k=5로 쌓은 library는 추가 probing 없이도 +0.9 / +4.4 / +5.1
- executor와 editor가 같은 모델이어도 모든 benchmark에서 개선 → 더 강한 editor가 필수는 아님

Case Study: JobBench Task (43.1 → 77.8)

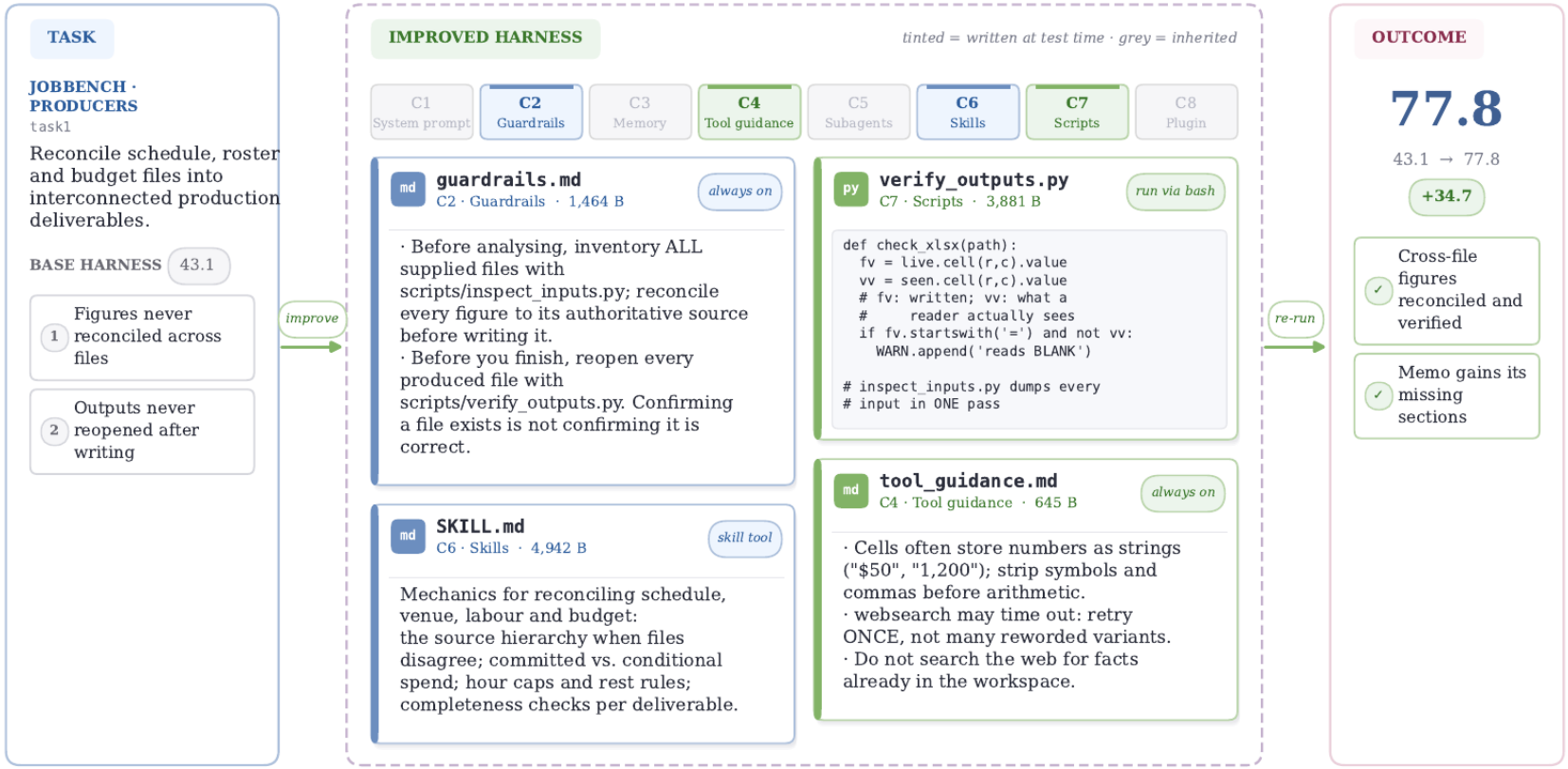


Figure 9. 회색 = 상속된 component, 색상 = test time에 새로 작성된 component.

실패 원인은 지식 부족이 아니라 절차 문제: ① 파일 간 수치 미대조 ② 산출물을 쓴 뒤 다시 열어 검증하지 않음. invariant edit(guardrails, verify script)와 domain edit(skill, tool guidance) 4개를 Composer가 모두 채택.

Conclusion & Impact

정답 없이도 harness를 스스로 개선하고, 그 개선 방법까지 학습할 수 있다

- task-level recursion: contrastive Propose–Probe–Compose로 verifier 없이 현재 task의 harness를 개선 (parallel/sequential로 scale)
- harnessing-level recursion: 개선 경험을 library에 쌓아 이후 task의 개선을 guide
- 세 benchmark, 두 executor, 두 initial harness에서 일관된 향상

Significance

- model weight를 건드리지 않는 online test-time adaptation의 방향 제시: 성능은 model뿐 아니라 harness 설계와 그 개선 과정에서도 나옴

Discussion Points

- behavioral contrast는 correctness 검증이 아님 (저자도 명시) → 잘못된 개선이 채택될 위험
- 보고된 수치는 저자들의 실험 결과이며, 기본 editor는 Claude Opus 4.8